Lecture-1

Goals, Roles and Myths of Software Engineering

Joys of Programming[Brooks, 1995]

```
- Bucuria de a crea lucruri care sunt si folositoare pentru altii
- Fascinatia de a rezolva probleme complexe
- Bucuria de a lucra intr un mediu flexibil
- Bucuria de a invata mereu
```

Woes of Programming[Brooks, 1995]

```
- Trebuie sa functioneze perfect
- Depinzi de altii si de implementarile lor
- Trebuie sa repari lucruri facute de altii, care intr-o lume ideala, ar fi
perfecte
- E greu sa gasesti bug uri
- Produsele se invechesc inainte de a fi gata
- > Legile Evolutiei Software
```

Software Crisis in 60's

```
- Cresterea puterii calculatoarelor si a problemelor care trebuie rezolvate
```

- E greu sa scrii programe care sa fie :
  1. Corecte
  2. Usor de inteles
  3. Usor de verificat/ testat

Root of Software Crisis

Complexity

```
- Cerinta problemelor/programelor care trebuiau implementate era mult mai
mare decat inainte
```

Expectations

```
- Preturile hardware ului scadeau, iar cele ale software ului cresteau
```

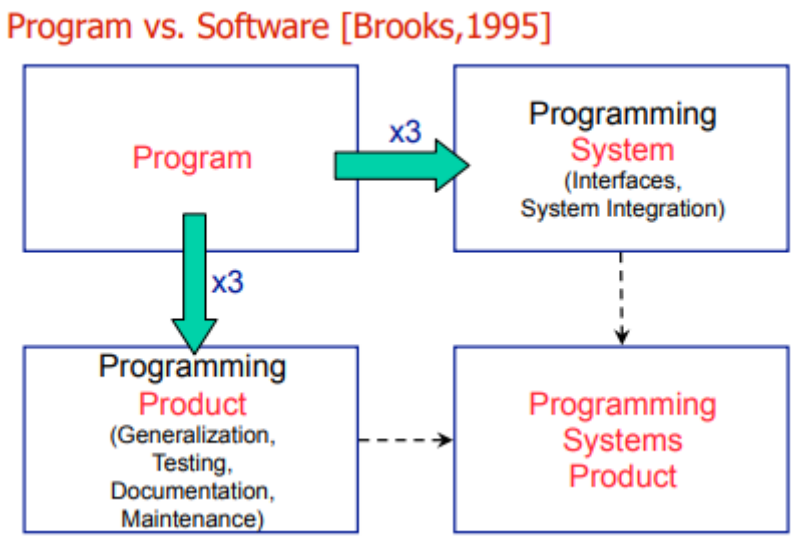
Change

```
- Nevoia de a mentine si de a updata programele
```

Small vs Large Projects

```
- Proiectele mici pot fi implementate usor de o persoana
- Cele mari au nevoie de impartirea sarcinii de munca, integrarea modulelor la care muncesc toti cei ce participa la proiect si management ul tuturor acestor lucruri
- Sistemele reale sunt mai mult decat cod
```

Program vs Software[Brooks, 1995]



- Exista doua moduri in care un program poate fi convertit intr-un produs
  - Programming Product : E un program pe care il poti rula, testa, repara si extinde indiferent de cine esti. E generalizat. Costa de 3 ori mai mult decat un program fara bug uri
  - Programming System : E o colectie de programe care interactioneaza, care impreuna sunt folosite pentru cerinte mai mari. Stiu din start cata memorie, spatiu vor lua.
- A Programming Systems Product -> costa de 9 ori mai mult decat un simplu program, insa e cel mai folositor obiect

What is Software engineering?

```
- Ingineria software are ca rol gestionarea complexitatea sistemelor
software
- Este disciplina care ia in considerare toate aspectele productiei Software
- Utilizeaza in mod adecvat tools uri si tehnicile pe care le are in functie
de:
    1. Problema pe care o ai(si pe care trebuie sa o rezolvi)
    2. Constrangerile de dezvoltare al produsului
    3. Resursele disponibile
```

What do we need to learn?

```
1. Processes : Cum sa gestionam proiecte de lunga durata
2. Techniques : Cum ar trebui sa abordam fiecare faza din proces
3. Tools : Cum am putea automatiza procesul pe cat de mult posibil
```

What is CASE (Computer-Aided Software Engineering)

```
Definitie : Sisteme software care intentioneaza sa ofere suport automat
pentru activitati din procesele software
```

Upper - CASE : Tools care te ajuta in early process activities (Design + requirements)
Lower - CASE : Tools care te ajuta in later activities cum ar fi programarea, debugging si testare

3 MAJOR QUESTIONS

What is Software?

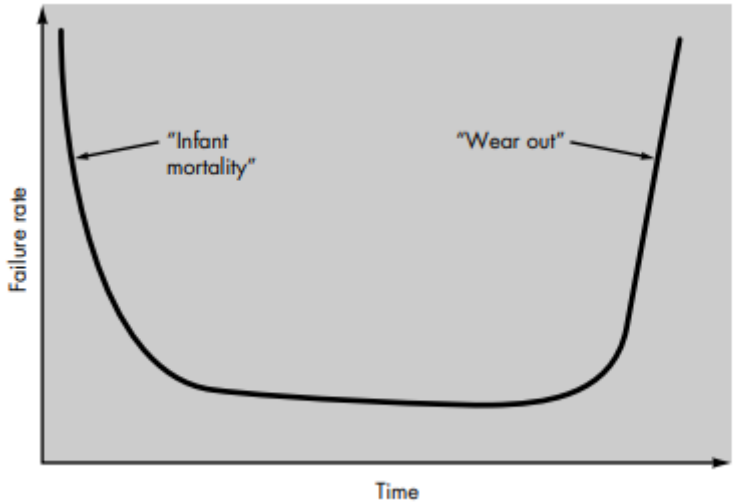
```
Computer programs si documente asociate: Requirements, design models and
user manuals
Produsele Software pot fi de doua tipuri:
    1. Generice : facute de companii mari, cu un anumit scop, pentru cat mai
multi clienti (ex : PC software such as Excel or Word)
    2. Custom : facute de niste firme software pentru un singur client, in
concordanta cu nevoile si cerintele acestuia
Noile software uri pot fi create prin :
    - dezvoltarea de noi programe (de la 0)
    - configurarea unor sisteme software deja existente(practic dai custom
la ceva deja facut, cum ar fi variante de aplicatii din China ca alternativa
la aplicatiile din restul lumii)
    - reutilizarea software urilor existente
```

Characteristics of Software

1. Software is engineered, not manufactured

```
- Software-ul e inginerie, nu manufactura -> lipseste costul material -> nu
e vorba de utilaje, nu e vb de tabla, sticla , siliciul pe care il folosesti
etc.
- Hardware ul este translatat intr o forma fizica(manufacturing)
- De ex a scazut volumul productiei de chip-uri -> nu se mai pot construi
atat de multe masini -concluzie : de asta e mai profitabil SW-ul
- Software ul este doar despre inginerie
- Manufacturing phase nu introduce probleme legate de calitate, dar nici
probleme care se corecteaza usor
- Costurile din SW vin din partea de engineering, mai exact de design, de
concept -> pana sa fie dat in productie
-un sistem SW nu se schimba cand se actualizeaza un algoritm, de ex un
sistem SW pt calculul salariilor -> daca se schimba legea trebuie doar sa se
actualizeze algoritmul care face calculul salariilor
```

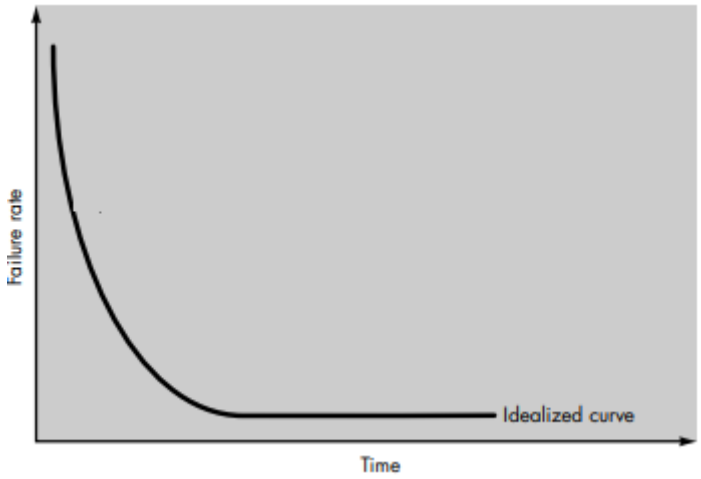
2. Hardware “Wears Out”. Software Doesn’t!



from **R.S.Pressman, 2005**

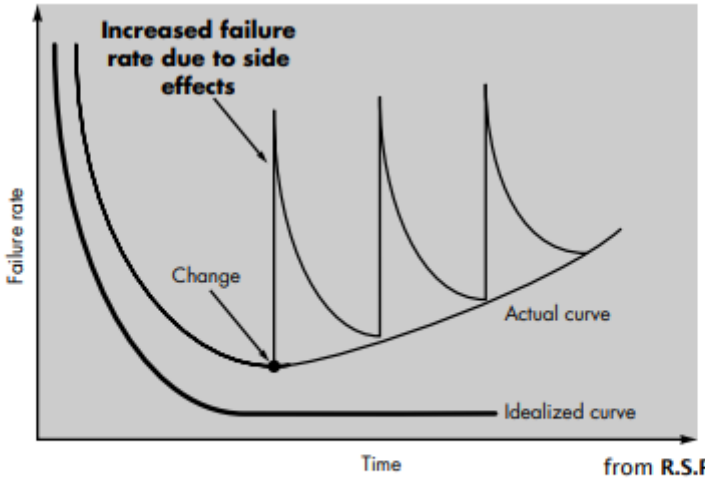
De exemplu, un televizor :
1. Daca nu merge din prima, e vb despre infant mortality -> Ceva e stricat
2. Daca merge, sansa sa se strice e mult mai mica (Inseamna ca a fost asamblat cum trebuie)
3. In timp (Wear out), televizorul se invecheste, componentele la fel, este ceva normal

IDEAL FAILURE CURVE FOR SOFTWARE



Contine partea de "Infant mortality" (primul spike) -> In timp, ne asteptam sa ramana minim nivelul de failure rate, deoarece, in comparatie cu hardware ul nu avem componente care sa imbatraneasca

ACTUAL FAILURE CURVE FOR SOFTWARE



Software does not “wear out”! But it does deteriorate!
The main cause for this phenomenon is **repeated change**

La inceput, e la fel, avem acelasi spike pentru "Infant mortality"

Dimensiunea și complexitatea unui sistem software vor crește continuu pe durata de viață. Tu trebuie mereu sa dai update cu noile cerinte la sistemul tau, pentru a evita ca sistemul sa devina obsolete si sa nu mai fie folosit.

Pe de altă parte, calitatea unui sistem software va scădea dacă nu este întreținut și adaptat riguros.

SW-ul poate imbatrani!!! De ce? Din cauza schimbarilor repetate

Daca dedicam timp pt mentenanta asta va influenta cat timp vom putea folosi sistemele SW respective

Un sistem SW functioneaza pe principiul “daca nu te schimbi, ai sa mori” -> sistemul trebuie sa se schimbe pt ca altfel devine nesatisfacator, problematic si inutil

Dar si daca te schimbi -> mori : orice schimbare a sistemului ii va creste complexitatea structurii si ii va scadea calitatea interna -> tot mai greu de intretinut si de lucrat cu el -> - curba/caldarea sistemului SW e in functie de masurile pe care noi le luam pt a imbunatatii structura interna
-> Se refera la punctul de "Change" -> unde avem o reiteratie a spike ului de "Infant mortality"

Nevoie sistemelor de a se schimba->caldarea cu rata erorilor in timp -> nu merge asimptotic la 0 din cauza ca trebuie sa schimbam sistemul din cand in cand -> in timp sistemele se erodeaza -> nu se mai pot folosi

Ideal Software Curve nu prea e relevant, deci nu are sens sa comparam realitatea cu ceva ce nu exista. Putem compara maxim ceva existent cu ceva existent

Nevoia de schimbare are deci 2 parti :
1. Enhance : Pentru a implementa noi cerinte
2. Extend & Adapt : Pentru a face programul **interoperable**(sa fie compatibil cu alte programe) si pentru a intalni nevoile noilor medii

Laws of Software Evolution

- **The Law of Continuing Change** (1974): Systems must be continually adapted else they become progressively less satisfactory.
- **The Law of Continuing Growth** (1980): The functional content of systems must be continually increased to maintain user satisfaction over their lifetime.

- **The Law of Increasing Complexity** (1974): As a system evolves its complexity increases unless work is done to maintain or reduce it.
- **The Law of Declining Quality** (1996): The quality of systems will appear to be declining, unless they are rigorously maintained and **adapted** to operational environment changes.

Software has no “sparse parts”

```
Sistemele SW sunt informatie, nu materie!!! -> nu e vb de manufacturing

Cand componentele hardware sunt depasite, ele pot fi inlocuite
Exemplu : la avion, fiecare piesa are un timp de uzura -> se inlocuieste
dupa un timp -> curba erorilor creste dupa un anumit timp

Nu avem sparse parts pentru software, esecul indica doar erori de design sau
in erori in procesul de producere al codului masina
**Mentenanta software este mult mai complexa decat mentenanta hardware**
```

3. Software is still custom- not component-built

In fiecare disciplina inginereasca, exista cateva componente standard create(de ex in hardware : circuite integrate)
In software avem nevoie de mai mult decat biblioteci algoritmice:
- reutilizarea algoritmilor si a structurilor de date
- ex : GUIs
Example of reusing in large: Frameworks : inversion of control: “Hollywood Principle”, adica :
- principiul inversarii controlului -> nu tu apelezi biblioteca, framework-urile te apeleaza pe tine -> principiul Hollywood -> call back : controlul e mereu la framewrok

What are the Costs?

Trebuie sa construim sisteme SW intr-o lume reala -> intrebarea aceasta care sunt consturile ne da framework-ul de constrangeri care trebuie indeplinit ca sa folosim sistemul nostrum SW in lumea reala
Costurile de mentenanta/intretinere a sistemului SW sunt mult mai mari decat cele de dezvoltare al lui

De obicei, costurile sistemului software este cel principal intr un computer system

Main Cost is **not** coding!

- System Tests are the strongest sequential constraint

Brook's Scheduling Rule:
1/3 Planning
1/6 Coding
1/4 Component Test
1/4 System Test

- Fraction devoted to planning is larger
- Half of schedule goes to testing

Not enough testing times is disastrous because bad news come late!

Cel mai mare cost nu e codarea, ci integrarea și testarea de sistem!!!! -> are cele mai mari constrangeri secventiale
Testarea are mai multe componente -> ele pot fi testate si individual -> daca una din piese intarzie, intarzie si testarea -> costuri mari
Planing -> captarea cerintelor si modelarea : toata partea de pregatire a task-urilor de codare -pe ultima parte vine testarea
Nu dureaza sa scriem cod, ci citind prin cod!!!
Trebuie sa petrecem cat mai mult timp pt testare ca sa pregatim sistemul SW pt client -> daca nu petrecem mult timp pe testare -> avem banana soft -> clientul incepe sa descopere multe bug-uri, ceea ce nu e ok
Testarea ia jumatate dn timp

What is good software?

Software Myths [Pressman, 2005]

Miturile sunt doar informatie gresita propagata
Sunt periculoase pentru ca :
1. Par intuitive si rezonabile
2. Sunt greu de schimbat

3 categorii de mituri :

1. Management myths
2. Customer myths
3. Developer myths

Management Myths

I. We have books, standards and procedures that guarantee good software

- ✓ Are they up-to-date? Are they used? Are they complete?

II. We buy high-end computers and most expensive CASE tools

- ✓ Are these used efficiently? Are these customized for specific needs?

III. Outsourcing will release us from software engineering problems

- ✓ Managing a project externally is harder than doing it internally

IV. If project gets late we will add more programmers and catch up

- ✓ F.P. Brooks calls this the "mythical man-month": *Adding people to a late project, makes the project even later*

Adding more programmers to a late project... [Brooks 95]

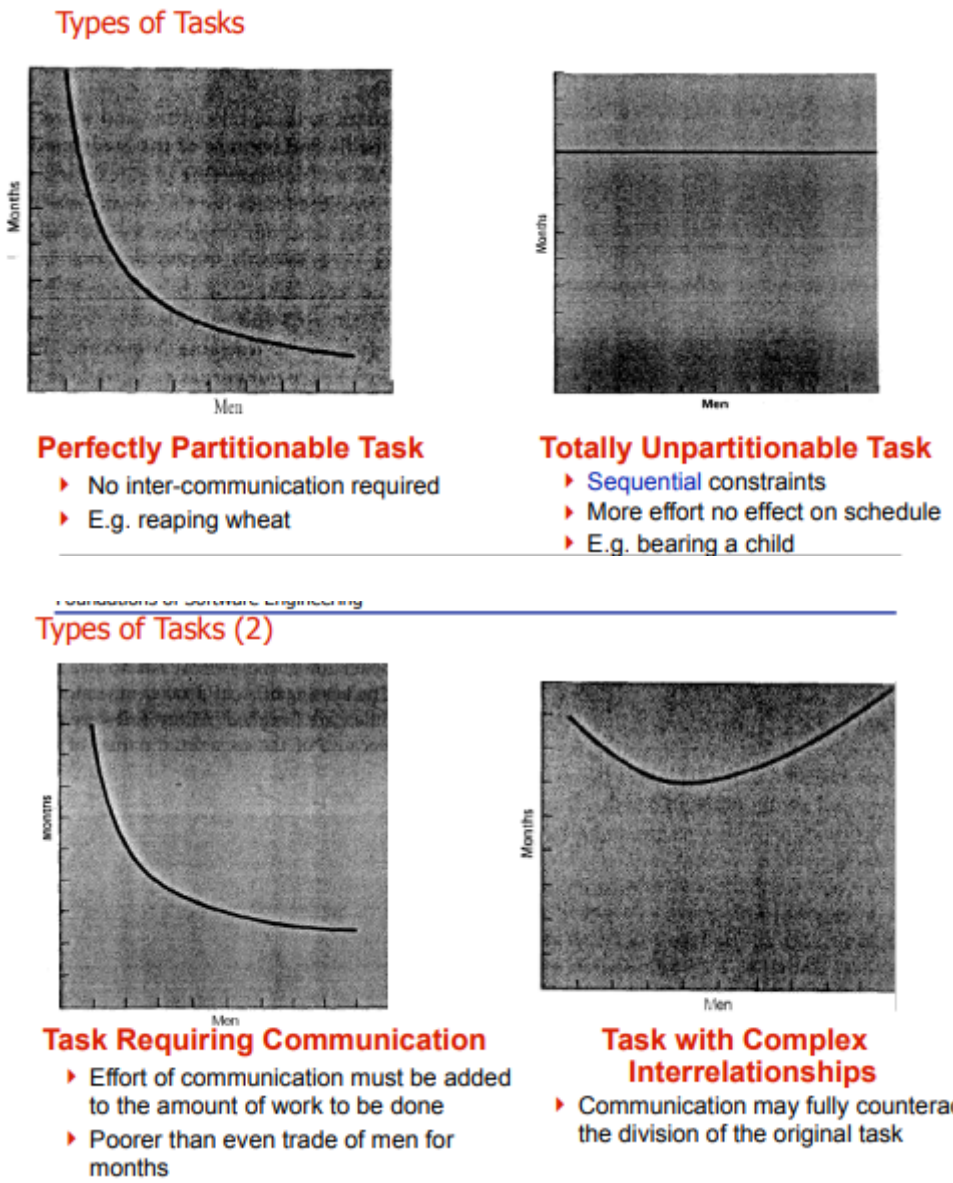
- Project is Planned for 3 people x 4 months
- If there is no time (e.g. we have to finish it in 2 months)…

What do we do??

- **ADD MANPOWER**
  - E.g. employ 3 more people
  - Believing that 3 people x 4 months = 6 people x 2 months

WRONG!

Pt ca exista diferite tipuri de task-uri : cu interdependente, perfect impartitionabila – nasterea unui copil, cele perfect partitionabile(cules mere). Oamenii noi trebuie sa inteleaga mai intai proiectul si abia apoi sa lucreze, lucru care ar lua timp



Communication

Training : Technological, goals of the effort, overall strategy plan of work
Intercommunication :

- For N people involved: N*(N-1)/2
- E.g. It is **6 x** in a team of 4 people compared to a team of 2 people

Brook's Law

Adding manpower to a late software project makes it later

```
Nr. of months of a project depends on its sequential constraints
Maximum nr. of people depends on the nr. of independent subtasks
```

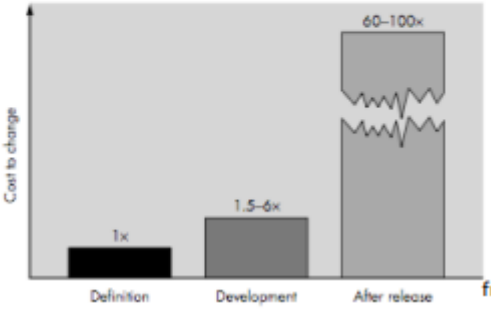
Daca task urile depind intre ele, nu poti sa faci mai rapid, pentru ca se va astepta finalizarea unui task
Nr de pers maxim pe care poti sa l adaugi intr-un proiect depinde de cate task uri independente ai

```
Legea lui Brooks : se formeaza din task-uri cu interdependente complexe : tu
trebuie sa termini un modul pt ca eu sa-l incep pe urmatorul si sa le putem
lega, tu trebuie sa testezi un modul ca sa putem testa apoi integrat cu un
alt modul etc. , trebuie sa se faca interfetele/API-urile ca sa le pot testa

Daca tot adaug oameni, pot scadea timpul de dezvoltare a SW-ului -> e foarte
periculos!!!! (la cules mere e periculos sa ai prea multi oameni pentru ca
se incurca intre ei dupa un punct)

Ideea de comunicare(orice interactiune intre participantii unui task) intr-o
echipa nu consta doar in vorbitul intre ei -> cu cat ai o echipa mai mare de
oameni, cu atat e mai greu de gestionat -> daca stau unii dupa altii e
foarte complicat

Legea lui Brooks : daca adaugi pe parcursul unui proiect SW in dezvoltare
mai multi oameni si proiectul e deja in intarziere, il faci sa intarzie si
mai mult: se creeaza mai multe interconexiuni -> implica un efort mult mai
mare, cei care intra mai tarziu trebuie sa depuna un efort mai mare ca sa
inteleaga conceptul

Parametrii dupa care se diseaca legea lui Brooks : timpul limitat, oamenii -
> timpul depinde de constrangerile secventiale -> sunt anumite lucruri pe
care nu le poti face in paralel, ci trebuie sa se execute secvential : de
ex, avem a, b , c etc. task-uri -> task-ul b nu poate fi indeplinit pana ce
task-ul a nu e indeplinit samd. (daca o femeie face un copil in 9 luni, 9
femei nu fac un copil intr-o luna)
Nr de oameni depinde de nr de task-uri independe pe care le poate gasi seful
-legea lui Brooks se leaga de miturile despre SW (mythical man fact)
Un program se transforma intr-un sistem SW cand e facut de mai multi oameni
pt mai multi oameni/client
```

Customer Myths

I. Just sketching the requirements is enough to start building software

II. Requirements can permanently change at a low cost because software is flexible



from **R.S.Pressman, 2005**

Aici ne gandim din perspectiva clientului.

After release e mai mult de schimbat

Developer Myths

I. Once we write a program and get it to work, our job is done

- ✓ *"the sooner you begin 'writing code' the longer it'll take to get done"*

II. The only deliverable work product is the working program

- ✓ program is only a part of a working configuration, which also includes documentation

III. Software engineering will make use create voluminous and unnecessary documentation, which slows us down

- ✓ Pressman: "*SWE is not about creating documents. It's about creating quality. Better quality leads to reduced work.*"

1. Mereu ai ceva de schimbat, tu trb sa te apuci sa scrii cod dupa ce ai inteles toate cerintele si dupa ce s a terminat tot planning ul
2. Trebuie sa avem documente(exemplu README git) pt SW -> documentam interfete, cod etc. -> ca sa putem sa scalam dezvoltarea SW-ului -> ca sa se faca fata codului -> inceput de a,b si c si sa fie preluat de x,y,z mai usor
3. Cum asiguri continuitatea sistemului SW? fix descris ca mai sus
   Personmounths e unitatea de măsura care se exprima in efortul depus de o un angajat full time intr-o luna